

MAPLE: Model-Aware Parameter Literature Extraction for QSP Model Calibration

Joel Eliason¹ and Aleksander Popel¹

¹Department of Biomedical Engineering, Johns Hopkins University, Baltimore, MD, USA

Abstract

Quantitative systems pharmacology (QSP) models require parameter values derived from experimental literature, yet current approaches face a trade-off between throughput and rigor. Both identifying relevant experiments and extracting data from them are labor-intensive: effective literature search requires understanding what parameters the model needs, what reactions they govern, and what experimental designs can constrain them. Manual curation remains the standard but produces inconsistent documentation, with uncertainty quantification and provenance tracked poorly if at all. Large language models (LLMs) can accelerate both literature search and extraction, but exhibit failure modes including paraphrasing source text (rather than quoting verbatim) and citation fabrication. We present MAPLE (Model-Aware Parameter Literature Extraction), a framework for QSP model calibration that combines LLM-assisted literature search and extraction with systematic validation. Model-aware prompts inject mechanistic context (parameter descriptions, reactions, rate laws) to guide LLM web search toward experiments that constrain specific parameters. A structured schema separates what the paper reports from how to use it for parameter calibration, capturing literature values with full provenance while constraining output to a machine-verifiable form. A multi-validator framework targets known LLM failure modes: value-in-snippet matching catches paraphrased text, DOI resolution flags fabricated citations, and reference consistency checks detect internal contradictions. The schema and validators also support manual curation without LLMs, enforcing structure and completeness. Validated targets are structured for downstream Bayesian inference. We evaluate MAPLE on calibration targets for a PDAC QSP model spanning tumor, stromal, and immune cell dynamics. We report validation pass rates, error types caught by each validator, and extraction runtime, characterizing where the workflow succeeds and where human

review remains necessary. MAPLE provides a reproducible pipeline from literature to calibrated QSP parameters while maintaining the provenance required for scientific applications.

1 Introduction

Quantitative systems pharmacology models translate mechanistic understanding of disease biology into predictions of drug response [Gadkar et al., 2016]. Capturing mechanistic detail across biological scales can require models with many components: some QSP models comprise tens to hundreds of ordinary differential equations representing cell populations, signaling molecules, and drug pharmacokinetics. Clinical endpoints are too sparse and aggregated to identify the individual parameters governing these dynamics; instead, values for proliferation rates, death rates, and binding affinities must come from targeted experimental studies reported in the literature. And because models are used for prediction, calibration must quantify uncertainty: virtual population generation and virtual clinical trials depend on parameter distributions, not point estimates [Rieger et al., 2018, Craig et al., 2023]. As the field increasingly integrates QSP with machine learning and artificial intelligence [Folguera-Blasco et al., 2024, Terranova et al., 2024], the bottleneck of manual parameter extraction becomes more acute: automated model development requires automated access to calibration data.

Beyond identifying relevant literature lies the challenge of extracting data systematically with full provenance. Each experiment reports results in different formats, with different units, time points, and uncertainty quantification. Experimental conditions vary across studies and are inconsistently documented. Converting a reported measurement into a calibration constraint requires judgment about applicability, uncertainty, and potential biases. The reasoning behind each extraction (why this value, from this paper, interpreted this way) must be documented for reproducibility and review.

The biological literature contains a wealth of experiments that isolate specific mechanisms from the complexity of in vivo systems. In vitro proliferation assays measure cell division rates without immune infiltration or nutrient gradients. Ex vivo cytotoxicity experiments quantify killing efficiency without the confounding effects of tumor heterogeneity or drug distribution. These isolated system experiments are valuable precisely because they remove confounding factors: when a

proliferation assay reports a doubling time, that measurement directly constrains the proliferation rate parameter. We use the term *calibration target* to refer to a structured representation of such experimental data, capturing what was measured, under what conditions, with what uncertainty, and how that measurement relates to model parameters.

Manual curation by domain experts remains the standard approach to extracting calibration targets from literature. An experienced modeler must understand what parameters the model needs, search for experiments that constrain those parameters, identify relevant passages, assess methodological quality, judge applicability, and document assumptions. The result, when done well, is a well-reasoned parameter estimate with clear provenance. But manual curation has significant limitations. The process is labor-intensive. Documentation quality varies across individuals and projects, with uncertainty quantification and provenance tracking handled inconsistently if at all. When personnel change, institutional knowledge disappears, and the reasoning behind a parameter choice may be lost even if the value persists in the model. A further challenge arises when combining constraints from multiple experiments: without a structured representation, joint inference requires ad-hoc scripting for each new combination of experiments, with no standard path from extracted data to inference code.

Automated extraction has been a goal of biomedical text mining for decades [Zhao et al., 2021], but traditional NLP pipelines struggle with the distributed, context-dependent information that characterizes experimental reports. Recent work has developed specialized NLP models for biomedical named entity recognition and relation extraction, with transformer-based approaches like BioBERT achieving high accuracy on domain-specific extraction tasks [Hu et al., 2024, Gao et al., 2024]. Deep learning approaches have also been applied to data element extraction for systematic literature reviews [Du et al., 2025]. For pharmacokinetic parameters specifically, active learning approaches have produced annotated corpora and fine-tuned language models for NER of PK terms [Gonzalez Hernandez et al., 2024]. However, these approaches require task-specific training data and do not generalize easily to the diverse parameter types encountered in QSP modeling.

Large language models offer a more flexible path forward. LLMs with web search capabilities can identify relevant literature when given appropriate context about what parameters need calibration and how they are used mechanistically in the model. Once papers are identified, LLMs can read

scientific text, follow structured templates, and extract information at scale. Recent work has demonstrated LLM-based extraction for biomedical knowledge graphs [He et al., 2026], systematic literature reviews [Khan et al., 2024], clinical endpoints [Gendrin-Brokmann et al., 2024], and QSP model development [Saini and Farnoud, 2025]. Dedicated software packages now exist for LLM-based biomedical information extraction [Hsu and Roberts, 2025], and interactive systems can extract structured data from scientific literature into structured formats [Wang et al., 2025].

But LLMs have well-documented failure modes that are particularly problematic for quantitative scientific work. Hallucination (the generation of plausible but incorrect information) is endemic to current models [Farquhar et al., 2024, Huang et al., 2025]. LLMs may fabricate numeric values, invent citations to papers that do not exist, or generate internally inconsistent outputs. These failures are especially dangerous because they often appear fluent and confident [Pandit et al., 2025]. Mitigation strategies include retrieval-augmented generation (RAG) and reasoning enhancement [Li et al., 2025], but for parameter extraction, where the extracted values directly influence model predictions, naive LLM extraction without validation is not acceptable.

Many of the failure modes of LLM extraction, however, are amenable to systematic detection. Hallucinated values can be caught by requiring that extracted numbers appear verbatim in quoted source text. Fabricated citations can be identified through DOI resolution. Internal inconsistencies can be flagged through schema validation. This suggests a workflow where LLMs handle tasks they do well (reading scientific text and following structured templates) while automated validators catch their characteristic errors before human review. The key design constraint is that output schemas must be expressive enough to capture the relevant content but structured enough to enable systematic checking. Pydantic [Colvin et al., 2025], a Python data validation library, provides this capability: when LLM output fails validation, the error message can be automatically fed back to the LLM for retry, creating a self-correcting loop that reduces the burden on human reviewers.

We present MAPLE (Model-Aware Parameter Literature Extraction), a framework for LLM-assisted calibration of QSP models with four components: (1) model-aware literature search, where prompts inject mechanistic context (parameter descriptions, reactions, rate laws) to guide LLM web search toward experiments that can constrain specific parameters; (2) a Pydantic-validated YAML schema that separates data extraction from model specification, constraining LLM output to a machine-verifiable form while preserving traceability from posterior to source text; (3)

a validation framework targeting specific failure modes, including value-in-snippet matching for hallucination detection and DOI resolution for citation verification; and (4) code generation for downstream Bayesian inference. We evaluate MAPLE on calibration targets for a PDAC QSP model, covering tumor, stromal, and immune cell dynamics, and report quantitative metrics on validation performance and extraction reliability.

2 Methods

2.1 Model-Aware Literature Search

The workflow begins with a parameter-driven search: the user specifies which model parameters need calibration, and the system builds a prompt that guides an LLM with web search capability to find relevant experimental literature. The framework supports multiple LLM providers and allows configuration of model version and reasoning effort (extended thinking), enabling users to trade off cost, latency, and extraction quality. A preview mode generates prompts without making API calls, allowing users to inspect and modify the prompt before committing to extraction.

The prompt injects mechanistic context extracted from the model structure. For each target parameter, the system automatically retrieves: the parameter’s name, units, and biological description; the reactions where the parameter appears with their rate laws; the species involved in those reactions with descriptions; other parameters appearing in the same reactions; and the broader reaction network showing other reactions that involve the same species. For example, when searching for data to calibrate a proliferation rate `k_apsc_prolif`, the prompt includes the reaction `aPSC -> 2*aPSC` with rate law `k_apsc_prolif * aPSC`, species descriptions explaining that aPSC represents activated pancreatic stellate cells, and related parameters like death rates that appear in the same reaction network.

This context guides the LLM’s search toward experiments that actually constrain the target parameters, rather than experiments that mention similar concepts but measure different quantities. The broader reaction network helps the LLM identify whether additional parameters should be jointly calibrated. For example, if a proliferation experiment also reports apoptosis data, the LLM can include the corresponding death rate parameter.

To encourage literature diversity, source exclusion prevents the LLM from reusing papers across

extractions. The prompt includes a list of primary sources already used for other calibration targets, and a strict exclusion mode extends this to sources used by related parameters in the same reaction network. This prevents the LLM from repeatedly extracting from the same well-known papers and promotes broader coverage of the experimental literature.

Beyond mechanistic context, the prompt includes the complete schema structure with field descriptions, enabling the LLM to generate valid output without trial and error. The prompt also provides explicit guidance on common pitfalls: distinguishing SEM from SD (with conversion formulas), avoiding hardcoded numeric values in code blocks, and properly documenting source relevance for cross-species or cross-indication extractions. By showing validation requirements upfront (such as the prohibition on non-peer-reviewed sources and required uncertainty thresholds for proxy data), the prompt reduces the retry loop by preventing predictable errors.

Once the LLM identifies a relevant paper, it extracts data into a structured schema that captures both the raw literature values and the modeling decisions needed for inference.

2.2 Schema Design

The SubmodelTarget schema structures calibration targets into two layers: an inputs layer that captures data exactly as reported in the literature, and a calibration layer that specifies how that data constrains model parameters. This separation reflects a data-first philosophy where extraction and model mapping are distinct operations.

2.2.1 The Inputs Layer

The inputs layer captures literature data with no modeling decisions. Each input includes:

- A unique identifier for reference elsewhere in the schema
- A numeric value with units
- Uncertainty specification: standard deviation or 95% confidence interval, when reported
- Sample size (n), for propagating uncertainty through the likelihood
- A type classification: direct measurement, proxy (requiring conversion), inferred estimate (interpreted from qualitative text), or assumed value

- A role designation: calibration target (included in the likelihood) or auxiliary (supporting data)
- A reference to a literature source with DOI
- A quoted text snippet from the source containing the value

The value snippet is particularly important. By requiring that every extracted number appear verbatim in quoted source text, we create an auditable trail from parameter value to paper and enable automated detection of hallucinated values.

Code block 1 shows a representative input extracted from literature.

```
inputs:
  - name: pdgf_fold_mean
    value: 4.37
    units: dimensionless
    uncertainty:
      sd: 0.49
    n: 3
    input_type: direct_measurement
    role: target
    source_ref: Schneider2001
    source_location: Abstract
    value_snippet: "Cell proliferation (4.37 +/- 0.49-
                    and 2.96 +/- 0.39-fold of control)."
```

Code block 1: An input capturing a proliferation measurement with provenance.

2.2.2 The Calibration Layer

The calibration layer specifies how extracted data constrains model parameters through a forward model that captures only the dynamics relevant to the experiment. For time-course data, this is typically a simplified ODE: a proliferation assay can be modeled as exponential growth ($dy/dt = k \cdot y$) where k is the proliferation rate. For steady-state or derived measurements, an algebraic model may suffice: if a paper reports half-life $t_{1/2}$, the forward model is simply $t_{1/2} = \ln(2)/k$,

183 requiring no ODE integration. In both cases, the submodel derives from the full QSP model by
184 isolating relevant species and treating confounding factors as absent.

185 The connection between submodel and full model is established through shared parameter
186 names. When a proliferation rate in a submodel is named `k_apsc_prolif`, it refers to the same
187 parameter in the full model. This naming convention allows posteriors from submodel inference to
188 inform full model calibration without post-hoc mapping. Because isolated experiments control con-
189 founding factors, the submodel captures exactly what the experiment measures, yielding posteriors
190 for parameters that are identifiable from that specific data.

191 The calibration layer specifies: which parameters to estimate, with prior distributions and
192 rationale; the forward model type (ODE template, algebraic formula, or custom code); observable
193 definitions linking model outputs to measured quantities; and likelihood specifications with explicit
194 references to inputs.

195 **Prior specification.** Each parameter includes a prior distribution with documented rationale.
196 The schema supports four distribution families: LogNormal for positive parameters where uncer-
197 tainty is multiplicative (common for rate constants); Normal for unconstrained parameters; Uni-
198 form for bounded parameters with no preference within the range; and HalfNormal for positive
199 parameters with mode at zero.

200 The rationale field documents the reasoning behind each prior choice, connecting literature val-
201 ues to distribution parameters and explaining the assumed uncertainty range. This documentation
202 serves both reviewers and future users who may need to modify the prior as new data becomes
203 available.

204 Code block 2 shows a parameter specification with prior and rationale.


```

calibration:
  parameters:
    - name: k_apsc_prolif
      units: 1/day
      prior:
        distribution: lognormal
        mu: 0.0      # log(1.0)
        sigma: 1.0
        rationale: |
          Wide prior centered at 1/day. A 4-fold increase
          in 1 day implies  $k \sim \ln(4) \sim 1.4/\text{day}$ .

```

Code block 2: A parameter with lognormal prior and documented rationale.

205 **Model types.** The schema provides built-in templates for common ODE patterns: exponential
 206 growth, first-order decay, two-state transitions, logistic growth, saturation dynamics, and Michaelis-
 207 Menten kinetics. For targets that do not require ODE integration, algebraic models specify closed-
 208 form relationships between parameters and observables (e.g., $t_{1/2} = \ln(2)/k$). When built-in types
 209 are insufficient, users can provide custom code in both Python (for validation) and Julia (for
 210 inference).

211 **Observable and error model.** The schema separates two distinct concerns: the *observable*
 212 specifies how to transform model state into a predicted measurement, while the *error model* specifies
 213 the statistical relationship between predictions and observed data.

214 The observable defines the mapping from forward model output to the quantity actually mea-
 215 sured in the experiment. For ODE-based models, this is either an identity mapping (the state
 216 variable is directly measured) or a custom transformation such as a ratio or fold-change. For alge-
 217 braic models, the forward model directly computes the observable, so no additional transformation
 218 is needed.

219 The error model specifies the likelihood function that connects predictions to data. It includes:
 220 the `observation_code` field containing Python code that derives the observed value and its un-
 221 certainty from extracted inputs; the likelihood distribution family (typically lognormal for positive

quantities, normal for unconstrained values); evaluation points indicating when predictions are compared to data (for ODE models); and sample size for uncertainty scaling.

The `observation_code` receives the inputs dictionary (with Pint quantities) and sample size, and returns a dictionary with: `value` (the point estimate), `sd` (the standard deviation for the likelihood), and `sd_uncertain` (a boolean indicating whether the SD itself should receive a hierarchical prior). This code runs during validation to verify consistency between raw inputs and derived statistics.

Code block 3 shows a forward model specification using the exponential growth template.

```
calibration:
  forward_model:
    type: exponential_growth
    rate_constant: k_apsc_prolif
    independent_variable:
      name: time
      units: day
      span: [0.0, 1.0]
    state_variables:
      - name: aPSC_rel
        units: dimensionless
        initial_condition:
          value: 1.0
          rationale: Normalized to control baseline.
    data_rationale: |
      BrdU assay measures proliferation over ~24h with
      constant PDGF stimulus. Exponential growth fits.
    submodel_rationale: |
      At saturating PDGF, the full model reduces to
       $dN/dt = k_{apsc\_prolif} * N$ .
```

Code block 3: Forward model specification with state variable and time span.

Code block 4 shows a complete error model entry. The `uses_inputs` list explicitly references which extracted inputs contribute to this measurement, enabling traceability from the likelihood back to source text. The `observable` block defines the transformation from state variables to

233 predicted measurement (here, an identity mapping). The `observation_code` derives the observed
234 value and uncertainty from the raw inputs, including the SEM-to-SD conversion. Finally, the
235 `likelihood` block specifies the distribution family. The error model is a list, allowing multiple
236 measurements per target when a single experiment reports data at several time points or conditions.

237 Code block 4 shows an error model specification.

```
error_model:
  - name: viability_48h
    units: dimensionless
    uses_inputs: [viability_mean, viability_sem]
    evaluation_points: [2.0] # days
    sample_size: 3
    observable:
      type: identity
      state_variables: [aPSC_norm]
    observation_code: |
      def derive_observation(inputs, sample_size, ureg):
          mean = inputs['viability_mean']
          sem = inputs['viability_sem'].magnitude
          sd = sem * np.sqrt(sample_size) # SEM to SD
          return {'value': mean, 'sd': sd}
    likelihood:
      distribution: normal
      rationale: |
        Viability near 1 with small SD; normal adequate.
```

Code block 4: Error model specification linking forward model predictions to observed data.

238 A complete example target is provided in Supplementary Section 1.

239 2.2.3 Source Documentation

240 Each target includes structured metadata about its data sources. The `primary_data_source` block
241 captures the main reference: DOI, title, authors, year, and a short `source_tag` used for cross-
242 referencing elsewhere in the schema. Secondary sources that provide supporting context (e.g.,

reviews explaining biological mechanisms, or related studies used for uncertainty estimation) are listed in `secondary_data_sources`, each with a `contribution` field explaining how that source informs the extraction.

2.2.4 Source Relevance Assessment

Experimental data rarely matches the target model perfectly. The `source_relevance` block documents the translation from source context to model context, making assumptions explicit and quantifying expected uncertainty.

The `indication_match` field classifies how well the source disease context matches the model: `exact` (same indication), `related` (same organ or disease class), `proxy` (different tissue used as mechanistic proxy), or `unrelated`. Cross-indication extractions require a `indication_match_justification` explaining why the data is still informative.

Species translation is captured by `species_source` and `species_target` fields. When these differ (e.g., mouse data informing a human model), the extraction must document the assumed translation.

The `source_quality` field categorizes the evidence type: `primary_human_clinical`, `primary_human_in_vitro`, `primary_animal_in_vivo`, `primary_animal_in_vitro`, `review_article`, or `textbook`. Non-peer-reviewed sources fail validation.

For immune and stromal parameters, `tme_compatibility` assesses whether the tumor microenvironment in the source matches the target model (`high`, `moderate`, or `low`). A T cell trafficking rate measured in melanoma (a T cell-permissive tumor) may not translate directly to PDAC (an immunosuppressive tumor); low compatibility requires explicit documentation.

Finally, `estimated_translation_uncertainty_fold` quantifies the expected uncertainty from all translation factors combined. Cross-species extraction from a proxy indication with low TME compatibility might warrant 10-fold or greater uncertainty, which propagates to prior width during inference.

2.2.5 Experimental Context

The `experimental_context` block captures the biological system where measurements were made: species (human, mouse, rat), system type (clinical resection, in vitro primary cells, cell line),

cell types with phenotype descriptions, and culture conditions for in vitro work (substrate, serum concentration, oxygen level). This metadata supports automated filtering of targets by context and helps reviewers assess whether the experimental system is appropriate for the modeling question.

2.2.6 Study Interpretation

Three free-text fields document the modeling rationale. The `study_interpretation` field summarizes how the experiment constrains the target parameters. The `key_assumptions` field lists modeling decisions (e.g., "exponential growth valid for early expansion," "negligible death during proliferation phase"). The `key_study_limitations` field documents known gaps between the experimental system and the model context (e.g., "2D culture lacks ECM stiffness present in vivo," "pharmacological stimulus may overestimate baseline rates"). These fields create a record of expert judgment that complements the quantitative extraction.

2.2.7 Schema Implementation

The schema is implemented using Pydantic [Colvin et al., 2025], a Python library for declarative data validation. Each schema component (inputs, parameters, forward models, error models, sources) is defined as a Pydantic model with typed fields and constraints. When YAML is parsed, Pydantic validates all constraints automatically: required fields must be present, values must match declared types, enumerations must use allowed values, and cross-field constraints must be satisfied.

Pydantic’s error messages are detailed and structured, reporting the field path, expected type, and actual value for each violation. This specificity is important for the LLM retry loop: when extraction fails validation, the error message provides actionable feedback. For example, a snippet validation error reports "Value-snippet mismatches (possible hallucination): input 'cell_count_mean' has value 4.37 but snippet contains '3.21'. Check that extracted values match the source text exactly." This tells the LLM both what failed and how to fix it.

The schema uses discriminated unions for forward model types. The `type` field determines which additional fields are required: an `exponential_growth` model requires `rate_constant`, while an `algebraic` model requires `code` and `code_julia`. This pattern ensures type-specific validation without manual dispatch logic.

Supplementary Section 5 provides the complete Pydantic model definitions.

2.3 Validation Framework

Beyond schema compliance, the validation framework comprises multiple automated checks targeting known LLM failure modes. Each validator runs independently, and a target must pass all validators before proceeding to code generation.

2.3.1 Reference Validation

LLMs sometimes generate references to entities that do not exist elsewhere in the document. Reference validators check that every `uses_inputs` reference matches a defined input name, every `source_ref` matches a defined source, parameters referenced in the model exist in the parameter list, and observable definitions reference valid state variables.

2.3.2 Content Validation

Content validators address fabrication of values, citations, and other factual claims.

DOI validation queries CrossRef for every DOI in the sources list. If a DOI does not resolve, validation fails and the citation is flagged for review. Title validation further checks that the extracted paper title fuzzy-matches the CrossRef metadata (using a 75% similarity threshold), catching cases where a valid DOI is paired with fabricated bibliographic details.

Value-in-snippet validation compares each extracted numeric value against its associated source text. If an input claims a value of 4.37 but the quoted snippet contains 3.21, validation fails. This is the primary anti-hallucination defense in the framework. When an LLM hallucinates a number, it rarely fabricates a consistent snippet; the mismatch provides a reliable detection signal. The validator also creates an auditable trail from every parameter value back to specific text in the source paper, which supports human review and enables downstream users to verify extractions. A separate external validator (described in Section 2.3.3) further verifies that snippets actually appear in the cited source papers, catching cases where the LLM fabricates plausible snippets wholesale.

Unit validation parses all unit strings using Pint [Grecco, 2025], a Python library for physical quantity handling. Malformed or unrecognized units trigger failure.

Code validation goes beyond syntax checking. The validator parses the code structure, then executes it with mock data to verify correct behavior. For `observation_code`, execution must return

the required dictionary keys (`value`, `sd`, `sd_uncertain`) with appropriate types. For observable code, execution must return arrays with correct units and dimensions. This catches logic errors and signature mismatches before inference.

Hardcoded constant detection scans code blocks for numeric literals that should be documented inputs. The parser identifies constants with units (e.g., `24 * ureg.hour`) that are not in the allowed set (small integers 0–5, common fractions, statistical percentiles). Each flagged constant must either be moved to an input with provenance or documented as an `observable_constant` with a `biological_basis` explanation.

Scale consistency validation executes observable code with mock species data spanning a 1000-fold range. If the output is constant despite varying inputs, or if the output magnitude suggests a ratio/score mismatch (e.g., values near 100 when 0–1 is expected), validation warns of potential unit or scaling errors.

Source relevance validators enforce the translation requirements described in Section 2.2.4. Non-peer-reviewed sources fail validation. Cross-species, cross-indication, and low TME compatibility extractions must specify minimum uncertainty thresholds (2-fold, 3-fold, and 10-fold respectively), ensuring that translation uncertainty propagates to prior widths.

Control character validation scans all string fields for characters that corrupt YAML parsing (ASCII control codes, non-breaking spaces, and similar). PDF copy-paste often introduces invisible characters that cause parsing failures; this validator catches them before they propagate.

Each validator raises a specific exception class from a hierarchy organized by failure mode: `SnippetValueMismatchError` for hallucinated values, `DOIResolutionError` for fabricated citations, `CodeExecutionError` for code that fails at runtime, `UnitParsingError` for invalid units, and so on. Each exception class carries a `category` attribute (hallucination, fabrication, code, units, reference, structural, prior) that enables aggregation across extraction runs. The exception messages are designed for the LLM retry loop, including both the specific failure and guidance for correction. This structured exception hierarchy also feeds into the extraction metrics reported in Section 3. Optional Logfire instrumentation [Pydantic, 2025] provides tracing of LLM calls and validation steps for debugging extraction failures.

2.3.3 External Validation

The validators above run automatically during schema parsing. An additional external validator verifies that value snippets actually appear in the cited source papers. This validator fetches paper text (abstract and full text when available) from Europe PMC and Unpaywall using the DOI, then uses fuzzy matching (80% similarity threshold) to locate each snippet in the retrieved text. Snippets that cannot be found in the source paper are flagged for review. This catches a subtle failure mode where the LLM fabricates plausible-looking snippets that contain the correct value but do not actually appear in the cited paper. Because this validation requires network access to external APIs, it runs as a separate manual step rather than during schema parsing.

2.3.4 Structural Validation

Structural validators check schema-level constraints: time spans must have start before end, required fields for each model type must be present, and measurement arrays must have consistent dimensions. Supplementary Section 3 provides implementation details for each validator.

2.4 Code Generation

2.4.1 Single-Target Translation

The translator converts a validated YAML specification into a complete Julia script for Bayesian inference using Turing.jl [Ge et al., 2018]. The generated script includes:

- Observed data constants extracted from the inputs layer
- A forward model function (ODE integration or algebraic computation)
- A Turing `@model` block defining priors for each parameter
- Likelihood terms connecting model predictions to observed measurements
- NUTS sampler configuration with default settings

Observed values are computed at translation time: the `observation_code` (Python) executes during script generation, and the resulting values become constants in the generated Julia script. This separates data extraction logic from inference logic.

Forward model handling depends on type. ODE-based models generate Julia functions for the DifferentialEquations.jl [Rackauckas and Nie, 2017] solver, with predefined templates for common dynamics and support for custom code. For exponential growth and first-order decay, analytical solutions replace numerical integration to reduce sampling overhead. Algebraic models bypass the solver entirely, computing observables directly from parameters.

The translator supports log-normal, normal, uniform, and half-normal priors. Measurement uncertainty can be fixed (constant in the likelihood) or estimated (hierarchical model with a prior on the noise level, triggered by `sd_uncertain: true`).

We chose to generate code rather than provide a library abstraction. Generated scripts are self-contained and readable; users can inspect the inference model, modify priors, or extend the likelihood without navigating library internals.

2.4.2 Joint Inference

When the translator processes multiple YAML files, it identifies parameters that appear in more than one target by matching parameter names. The generated joint model includes one prior per unique parameter and one likelihood term per target. Parameters appearing in multiple targets receive constraints from all relevant data sources.

The mechanics of parameter sharing are straightforward. Consider two targets A and B that both involve a parameter `k_shared`. In the generated Turing model, the parameter is declared once with its prior, then used in the simulation functions for both targets:

```
@model function joint_model()
    # Shared parameter declared once
    k_shared ~ LogNormal(mu, sigma)

    # Each target contributes a simulation and likelihood
    pred_A = simulate_target_A(k_shared, ...)
    pred_B = simulate_target_B(k_shared, ...)

    obs_A ~ Normal(pred_A, sigma_A)
    obs_B ~ Normal(pred_B, sigma_B)
end
```

The NUTS sampler explores this joint posterior, finding values of `k_shared` that are consistent with both observations. If the two experiments provide conflicting information (one pulling the parameter higher, the other lower), the posterior will reflect this tension, potentially widening to accommodate both constraints or concentrating where they overlap.

Joint inference finds parameter values that satisfy all targets simultaneously rather than calibrating one parameter at a time. When a parameter appears in multiple experiments with different designs, both likelihoods update the same posterior, yielding tighter constraints than either alone could provide. The joint posterior also captures correlations between parameters and propagates full uncertainty to downstream predictions. Supplementary Section 4 provides details on the generated code structure.

2.5 Posterior Diagnostics

After running the generated inference scripts, we assess inference quality using standard Bayesian diagnostics.

Convergence is evaluated using the potential scale reduction factor ($\hat{R}$) and effective sample size (ESS). An $\hat{R}$ near 1.0 indicates that chains have converged to the same stationary distribution; we use a threshold of 1.01. ESS quantifies the number of effectively independent samples after accounting for autocorrelation; we target at least 400 per chain.

Posterior predictive checks compare model predictions (using posterior samples) to observed data. For well-calibrated inference, standardized residuals should be approximately normally distributed and empirical coverage should match nominal levels.

Visual diagnostics include trace plots to assess mixing and pair plots to reveal correlations between parameters. Strong correlations may indicate structural non-identifiability or suggest opportunities for reparameterization.

2.6 Application: PDAC QSP Model

We evaluate MAPLE on 18 parameters from a pancreatic ductal adenocarcinoma QSP model, spanning tumor growth (proliferation rate, initial diameter), stromal dynamics (PSC activation, death rates for quiescent and activated states), ECM turnover (secretion and degradation rates),

immune cell recruitment (macrophages, MDSCs, CD8+ T cells, regulatory T cells), macrophage polarization, chemokine secretion (CCL2), and T cell biology (IL-2 thresholds, clonal diversity).

2.6.1 Evaluation Metrics

We report metrics across four stages of the workflow:

Extraction performance. First-attempt validation pass rate, retry distribution, average extraction time, and cost per target (API tokens and dollars).

Error analysis. Distribution of validation failures by category (hallucination, fabrication, units, code, prior specification), and tool usage patterns (web search and code execution calls).

Source characteristics. Distribution of source types (primary human, primary animal, review), species translation requirements (human→human vs. cross-species), and indication match (exact, related, or proxy).

Inference quality. Convergence diagnostics ($\hat{R}$, effective sample size), posterior summaries (medians and 90% credible intervals), and consistency across targets constraining the same parameter.

3 Results

3.1 Extraction and Validation

We applied MAPLE to 18 parameters from a PDAC QSP model, covering tumor growth, stromal dynamics, immune recruitment, and cytokine signaling. All 18 targets were successfully extracted (Table 1).

None of the extractions passed validation on the first attempt; all required at least one retry. Seven targets succeeded after a single retry, while parameters like CCL2 secretion and ECM production required up to 7 iterations to resolve unit ambiguities or reconcile conflicting literature. Each extraction took 7.9 minutes on average, costing \$0.37 per target (\$6.72 total).

The validators caught 24 errors during extraction (Table 2). Unit errors were the most common, reflecting inconsistent unit conventions across experimental papers. Fabrication errors occurred

Table 1: Extraction metrics per calibration target.

Target	Retries	Duration (min)	Tokens	Cost
N_IL2_CD4	3	6.5	198,267	\$0.33
N_IL2_CD8	1	3.4	98,233	\$0.19
initial_tumour_diameter	1	3.0	70,219	\$0.15
k_C1_growth	1	4.1	81,606	\$0.19
k_CCL2_sec	5	9.2	367,754	\$0.54
k_ECM_apsc_sec	5	10.8	285,309	\$0.57
k_ECM_deg	1	5.8	135,317	\$0.30
k_ECM_qpsc_sec	5	12.0	406,891	\$0.56
k_M1_pol	1	7.1	190,149	\$0.40
k_MDSC_rec	3	8.4	221,048	\$0.42
k_Mac_rec	3	10.0	239,265	\$0.42
k_apsc_death	2	5.4	134,626	\$0.26
k_psc_activation	7	9.9	354,840	\$0.47
k_psc_const	2	7.7	209,064	\$0.44
k_qpsc_death	1	3.8	91,974	\$0.18
n_CD8_clones	1	5.3	114,839	\$0.27
n_Treg_clones	2	5.7	133,558	\$0.28
q_Treg_T_in	6	23.9	586,283	\$0.77
Average	2.8	7.9	217,735	\$0.37

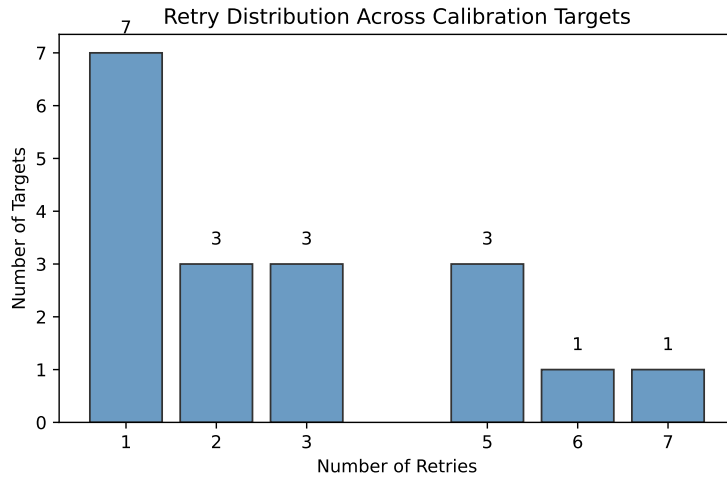


Figure 1: Distribution of retry counts across 18 extraction attempts.

when the LLM generated plausible but invalid DOIs, which the DOI resolution validator rejected. Prior specification errors arose when proposed uncertainty ranges did not match the stated trans-
lution uncertainty. Code errors were caught when observation functions failed unit checks.

Table 2: Error categories encountered during extraction. Each target may have multiple errors.

Error Category	Count	%
units	9	38%
prior	5	21%
fabrication	4	17%
code	4	17%
other	1	4%
hallucination	1	4%
Total	24	

The LLM used 210 web searches to find and verify sources, and 22 code executions to check observation functions.

3.1.1 Source Characteristics

The extracted targets drew from a range of experimental sources (Table 3). Human clinical data accounted for 38% of targets, with the remainder from animal studies (in vivo and in vitro).

Table 3: Distribution of primary data source quality across calibration targets.

Source Quality	Count	%
primary_human_clinical	7	39%
primary_animal_in_vitro	5	28%
primary_animal_in_vivo	4	22%
primary_human_in_vitro	2	11%
Total	18	

About 44% of targets used human data directly (Table 4). The rest drew from mouse (33%) or rat (11%) experiments, which would require cross-species scaling before use in a human QSP model.

Only 27% of targets came from PDAC-specific experiments (Table 5). Most (61%) relied on proxy indications: other cancer types, chronic pancreatitis, or fibrosis models with similar biology.

Table 4: Species translation from data source to model target. Human→human indicates no cross-species translation required.

Translation	Count	%
human→human	8	44%
mouse→human	6	33%
rat→human	2	11%
human_and_rat→human	1	6%
chick→human	1	6%
Total	18	

Table 5: How closely the data source indication matches the model target (PDAC).

Indication Match	Count	%
Related indication (e.g., other cancers, fibrosis)	11	61%
Data from PDAC patients/models related	5	28%
	2	11%
Total	18	

3.2 Code Generation

All 18 targets were translated to a joint Julia inference script. The translator identified 18 unique parameters and generated corresponding priors and likelihood terms. Most targets (94%) use closed-form forward models; the rest use simple ODEs (Table 6).

Table 6: Distribution of forward model types used in calibration submodels.

Model Type	Count	%
Algebraic (closed-form)	17	94%
Two-state ODE	1	6%
Total	18	

The generated script is self-contained, requiring only DifferentialEquations.jl, Turing.jl, and Distributions.jl [Besançon et al., 2021]. Each target contributes a simulate function, observed data constants, and a likelihood term. The complete script is provided in the supplementary materials.

In this application, each target constrains a single isolated parameter with a closed-form forward model. For such cases, joint Bayesian inference is not strictly necessary; one could simply invert each forward model and propagate measurement uncertainty. The framework is designed for more complex scenarios: when multiple derivations constrain the same parameter, when a single target

constrains multiple parameters, or when forward models involve ODEs that cannot be analytically inverted.

3.3 Convergence Diagnostics

We ran 4 chains of 1000 samples each using the NUTS sampler. All parameters achieved $\hat{R} < 1.01$ and effective sample sizes above 3000 (Table 7). Sampling took about 42 seconds on an M-series MacBook Air.

Table 7: Convergence diagnostics for MCMC sampling. All $\hat{R} < 1.01$ indicates convergence.

Parameter	$\hat{R}$	ESS (bulk)	ESS (tail)
N_IL2_CD4	1.001	4562	2718
N_IL2_CD8	1.001	3341	2004
k_C1_growth	1.000	4375	2928
k_CCL2_sec	1.003	4290	2874
k_M1_pol	1.003	4501	2692
k_MDSC_rec	1.000	4973	3006
k_Mac_rec	1.005	4958	2867
k_apsc_death	1.004	4608	2759
k_psc_activation	1.001	4942	2653
n_CD8_clones	1.001	4682	2681
q_CD8_T_in	1.002	4864	2782
q_Treg_T_in	1.002	4688	2839

3.4 Posterior Distributions

Figure 2 shows posterior estimates for all 18 parameters. Table 8 reports medians and 90% credible intervals.

Most posteriors are narrower than their priors, indicating that the data constrained the parameters. The tumor growth rate **k_C1_growth** ($5.37\text{e-}03 \text{ day}^{-1}$, 90% CI: $2.24\text{e-}03\text{--}0.0127$) implies a doubling time of about 130 days, consistent with clinical PDAC observations. The activated PSC death rate **k_apsc_death** (0.0707 day^{-1}) implies a half-life of about 10 days.

The IL-2-dependent division counts for CD4+ T cells (7.99) fall within the 7–8 generation range expected from CFSE proliferation tracking, where dye dilution limits counting. The CD8+ estimate (5.3) is somewhat lower, possibly reflecting differences in experimental conditions between the source studies.

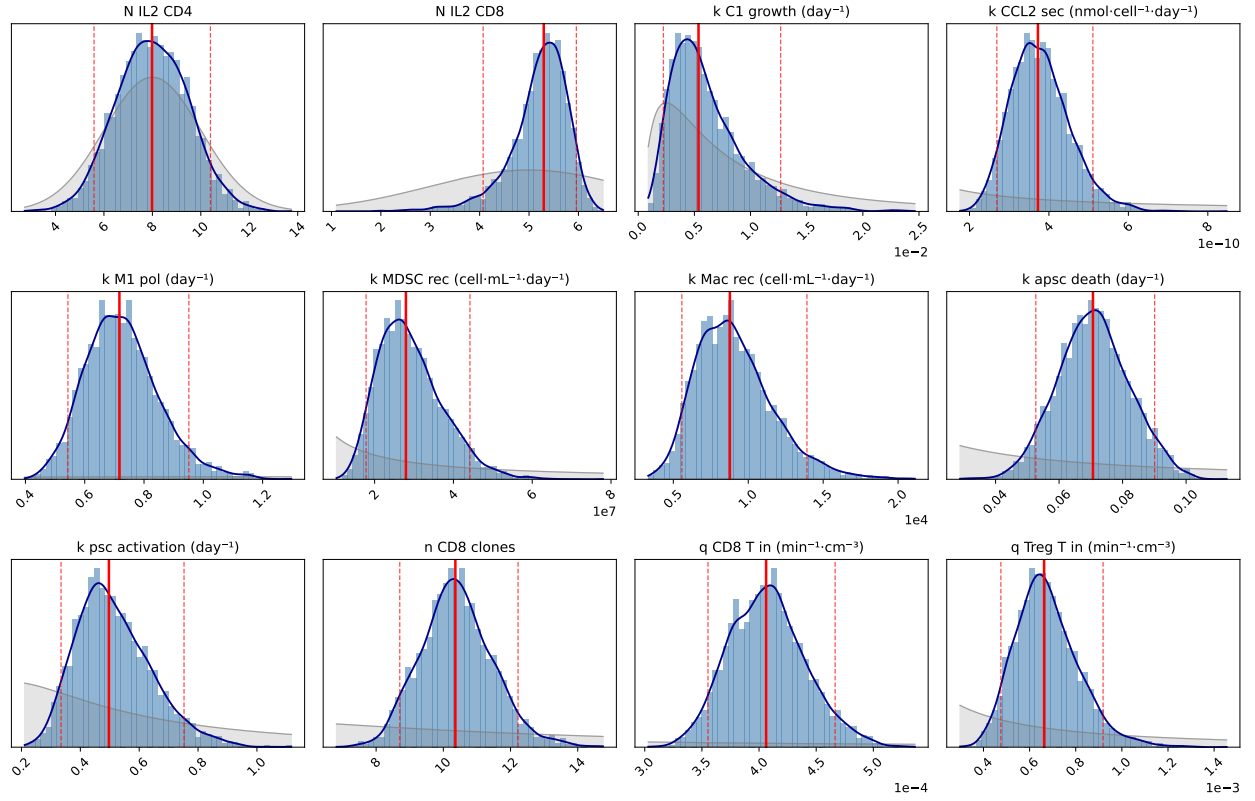


Figure 2: Posterior distributions from joint Bayesian inference. Gray shading shows the prior distribution; blue histogram and line show the posterior. Red lines indicate median (solid) and 90% credible interval bounds (dashed).

Table 8: Posterior parameter estimates from joint Bayesian inference. Credible intervals are 90% highest density intervals. [†]Auxiliary parameter introduced during extraction.

Parameter	Median	90% CI	Units
N_IL2_CD4	7.99	[5.59, 10.4]	—
N_IL2_CD8	5.3	[4.07, 5.96]	—
k_C1_growth	5.37e-03	[2.24e-03, 0.0127]	day ⁻¹
k_CCL2_sec	3.72e-10	[2.69e-10, 5.11e-10]	nmol·cell ⁻¹ ·day ⁻¹
k_M1_pol	0.717	[0.543, 0.951]	day ⁻¹
k_MDSC_rec	2.79e+07	[1.78e+07, 4.42e+07]	cell·mL ⁻¹ ·day ⁻¹
k_Mac_rec	8.77e+03	[5.57e+03, 1.39e+04]	cell·mL ⁻¹ ·day ⁻¹
k_apsc_death	0.0707	[0.0527, 0.0901]	day ⁻¹
k_psc_activation	0.497	[0.334, 0.752]	day ⁻¹
n_CD8_clones	10.4	[8.7, 12.2]	—
q_CD8_T_in	4.06e-04	[3.55e-04, 4.67e-04]	min ⁻¹ ·cm ⁻³
q_Treg_T_in	6.63e-04	[4.75e-04, 9.19e-04]	min ⁻¹ ·cm ⁻³

4 Discussion

MAPLE demonstrates that LLM-assisted parameter extraction can achieve both reasonable throughput and the rigor required for QSP model calibration. The key insight is that LLM failure modes are systematic and detectable: hallucinated values rarely match quoted source text, fabricated DOIs fail resolution, and malformed code fails execution. By targeting these failure modes with specific validators, the framework catches errors automatically and uses structured feedback to guide correction.

4.1 Validation as the Core Mechanism

None of the 18 extractions passed validation on the first attempt. This is not a limitation but rather the expected behavior of the system. The validators serve as an automated first-pass review, catching errors that would otherwise require human attention. The retry loop, where validation failures are fed back to the LLM with specific error messages, resolves most issues without human intervention. The extractions that required multiple retries (up to 7) typically involved ambiguous unit conventions or conflicting literature values, situations where the LLM needed multiple attempts to find consistent interpretations.

The value-in-snippet requirement proved particularly effective. By requiring that every extracted number appear verbatim in quoted source text, we create an auditable trail and catch a common LLM behavior: paraphrasing. The LLM typically extracts accurate content on the first attempt, but rephrases source text rather than quoting it exactly. The validator flags these cases, and the retry loop produces verbatim snippets. A separate external validator then checks that snippets actually appear in the cited papers, catching cases where the LLM generates plausible text that does not exist in the source. Together, these validators support human review: a reviewer can verify each extraction by checking the snippet against the source paper, without re-reading entire methods sections.

4.2 Schema Design Choices

The separation of data extraction from model specification reflects a deliberate design philosophy. The inputs layer captures what the paper reports, with no modeling decisions; the calibration

layer specifies how that data constrains parameters. This separation has several benefits. First, it clarifies where human judgment is required: the inputs should be objective (what numbers appear in the paper), while the calibration layer involves modeling choices (what forward model to use, what prior width is appropriate). Second, it enables reuse: if a modeling assumption changes, the inputs remain valid and only the calibration layer needs revision. Third, it supports review: a domain expert can verify inputs by checking source text, while a modeler can assess the calibration specification independently.

The forward model abstraction similarly separates concerns. Built-in templates for common ODE patterns (exponential growth, first-order decay) reduce the opportunity for error in routine cases. Custom code blocks provide flexibility for complex situations while remaining subject to validation. The choice to generate complete Julia scripts rather than provide a library abstraction prioritizes transparency: users can inspect exactly what inference will run, modify it if needed, and debug issues without navigating abstraction layers.

4.3 Limitations of Extracted Data

The source characteristics reveal a fundamental challenge in QSP model calibration. Only 27% of targets came from PDAC-specific experiments; the majority relied on proxy indications. Similarly, 33% of targets required cross-species translation from mouse data. These are not failures of the extraction process but rather reflections of available literature: PDAC-specific experiments measuring the exact quantities needed for model calibration are rare.

The schema addresses this reality through explicit documentation. The `source_relevance` block requires that cross-species and cross-indication extractions specify translation uncertainty, which propagates to prior widths during inference. This does not solve the underlying data gap, but it makes assumptions explicit and quantifies their impact on parameter uncertainty. A parameter calibrated from mouse fibrosis data with 5-fold translation uncertainty will have a correspondingly wide posterior, appropriately reflecting the limited applicability of available evidence.

4.4 Comparison to Manual Curation

Manual curation by domain experts remains the gold standard for parameter extraction quality. An experienced modeler brings biological intuition, can assess methodological quality, and can

make nuanced judgments about applicability. MAPLE does not replace this expertise but rather restructures where it is applied.

In manual curation, the expert performs all tasks: literature search, data extraction, quality assessment, unit conversion, uncertainty estimation, and documentation. This conflates tasks with different skill requirements and error profiles. Literature search benefits from domain knowledge; numeric extraction is tedious and error-prone; documentation is often neglected under time pressure.

MAPLE shifts the expert’s role toward review and validation. The LLM handles literature search (guided by model-aware prompts) and structured extraction (guided by schema requirements). The expert reviews extracted targets, assesses source relevance, and approves or corrects the calibration specification. This division leverages LLM strengths (reading many papers quickly, following structured templates) while preserving human judgment where it matters most.

The structured schema also addresses a persistent problem in manual curation: documentation quality. When parameter values are extracted manually, the reasoning is often recorded incompletely or not at all. The schema enforces documentation: every input requires a source reference and snippet, every prior requires a rationale, every cross-indication extraction requires a justification. This creates a record that survives personnel changes and enables future review.

4.5 Relationship to Existing Approaches

MAPLE addresses a specific gap in the landscape of LLM-assisted extraction tools. General-purpose biomedical extraction frameworks like LLM-IE [Hsu and Roberts, 2025] provide building blocks for named entity recognition and relation extraction, but do not address the downstream requirements of QSP calibration: uncertainty quantification, forward model specification, and code generation for inference. Interactive systems like SciDaSynth [Wang et al., 2025] support structured data extraction with human-in-the-loop refinement, but focus on tabular output rather than the hierarchical schema needed for Bayesian calibration. QSP-Copilot [Saini and Farnoud, 2025] applies LLMs to QSP model development broadly, including literature extraction, but emphasizes model structure discovery (entity-interaction pairs) rather than quantitative parameter calibration with provenance.

The validation approach in MAPLE draws on established strategies for hallucination detection. Requiring extracted values to appear in quoted source text is analogous to the grounding strategies

used in retrieval-augmented generation [Li et al., 2025], where LLM outputs are verified against retrieved documents. The schema-based validation enforces structural constraints similar to those in constrained decoding and structured output systems, where JSON schemas guide LLM generation toward valid output [Geng et al., 2025]. The key contribution is combining these strategies with domain-specific validators (DOI resolution, unit parsing, code execution) in a framework tailored to quantitative scientific extraction.

Recent work on knowledge graph construction from biomedical literature [He et al., 2026, Gao et al., 2024] demonstrates that LLMs can extract structured relationships at scale. The Immune Cell Knowledge Graph (ICKG) used zero-shot prompting with Llama 3.1 to extract activation and inhibition relationships from over 24,000 PubMed abstracts. Such approaches prioritize coverage and can tolerate some extraction errors because downstream analyses aggregate across many relationships. Parameter calibration has different requirements: each extracted value directly influences model predictions, so individual errors matter more. MAPLE trades some throughput for higher per-target validation stringency.

The IQ Consortium has outlined best practices for AI/ML in quantitative modeling, including the use of NLP for knowledge extraction from biomedical literature [Terranova et al., 2024]. They emphasize explainability, human-in-the-loop oversight, and consideration of context of use. MAPLE aligns with these principles: the schema documents reasoning and provenance, validators catch errors before human review, and the generated code is transparent and modifiable. The growing interest in coupling QSP with machine learning [Folguera-Blasco et al., 2024, Pawłowski et al., 2024] suggests that hybrid approaches combining mechanistic modeling with data-driven methods will become increasingly important; automated literature extraction is one component of this broader integration.

4.6 Limitations and Future Work

Several limitations merit discussion. First, the framework currently relies on the LLM’s built-in web search to find and access source literature. This means extraction quality is bounded by the LLM’s ability to locate relevant papers and read their content. Closed-access publications may be inaccessible or only partially available (e.g., abstract only), limiting both source discovery and the depth of extraction. Future work should integrate retrieval-augmented approaches that provide the

LLM with full-text access to institutional library holdings, enabling extraction from deeper within papers rather than relying on what the LLM can retrieve through web search alone. More broadly, the framework depends on LLM capabilities that continue to evolve: extraction accuracy depends on model capabilities, cost and latency affect practical usability, and extraction is stochastic, with the same query potentially yielding different papers and extractions across runs. The modular architecture allows swapping LLM providers, model versions, and reasoning effort settings, but performance characteristics may differ.

Second, human review remains essential. The validators catch many errors automatically, but they cannot assess biological plausibility or methodological quality. A paper may report a measurement correctly, but that measurement may be unreliable due to experimental limitations not evident from the abstract. Domain expertise is required to assess whether a source is appropriate for the modeling question. In practice, a hybrid workflow may be most effective: use the LLM to gather data and produce a structured draft, then manually edit the YAML. The schema and validators can also be used without LLMs at all, providing structure and completeness checking for manual curation.

Third, the current evaluation focuses on isolated-system experiments where confounding factors are controlled. The schema is general enough to represent more complex data (time courses, dose-responses, multi-compartment studies), but, in our experience, LLMs are not yet reliable at extracting structured quantitative data from figures or tables in papers. Many QSP parameters require this kind of complex data, and automated extraction remains a challenge.

Finally, the evaluation presented here covers a single model (PDAC QSP) and a limited set of parameters. Broader evaluation across different therapeutic areas and model structures would better characterize where the framework succeeds and where it struggles.

Study Highlights

What is the current knowledge on the topic?

QSP model calibration requires parameter values from experimental literature, but manual curation is labor-intensive with inconsistent documentation practices, often lacking systematic uncertainty quantification and provenance, while LLM extraction exhibits hallucination and fabrication

errors that are unacceptable for quantitative modeling.

What question did this study address?

Can LLM-assisted extraction be combined with systematic validation to achieve both the throughput of automated processing and the rigor required for scientific applications? Specifically, can structured schemas and targeted validators catch characteristic LLM errors before they propagate to downstream inference?

What does this study add to our knowledge?

We demonstrate that LLMs with web search can identify relevant experimental literature when given mechanistic context about target parameters. Requiring extracted values to appear verbatim in quoted source text provides an effective check against hallucination. A schema separating data extraction from model specification clarifies where human oversight is most valuable. Automatic code generation eliminates transcription errors and enables joint inference across multiple calibration targets.

How might this change drug discovery, development, and/or therapeutics?

MAPLE provides a reproducible pipeline from literature to calibrated QSP model parameters. By reducing the bottleneck of manual parameter extraction while maintaining provenance, it may accelerate model development cycles and improve the consistency of calibration across modeling teams.

Acknowledgments

Funding

Conflict of Interest

The author declares no conflict of interest.

Data Availability Statement

MAPLE (the SubmodelTarget schema, validation framework, and Julia code generator) is available at <https://github.com/popellab/qsp-llm-workflows>. The ten PDAC calibration targets used

in this study are provided as supplementary YAML files. Generated Julia inference scripts are included in the supplementary materials.

References

K Gadkar, DC Kirouac, DE Mager, PH van der Graaf, and S Ramanujan. A Six-Stage Workflow for Robust Application of Systems Pharmacology. *CPT: Pharmacometrics & Systems Pharmacology*, 5(5):235–249, May 2016. ISSN 2163-8306. doi: 10.1002/psp4.12071.

Theodore R. Rieger, Richard J. Allen, Lukas Bystricky, Yuzhou Chen, Glen Wright Colopy, Yifan Cui, Angelica Gonzalez, Yifei Liu, R. D. White, R. A. Everett, H. T. Banks, and Cynthia J. Musante. Improving the generation and selection of virtual populations in quantitative systems pharmacology models. *Progress in Biophysics and Molecular Biology*, 139:15–22, November 2018. ISSN 0079-6107. doi: 10.1016/j.pbiomolbio.2018.06.002.

Morgan Craig, Jana L. Gevertz, Irina Kareva, and Kathleen P. Wilkie. A practical guide for the generation of model-based virtual clinical trials. *Frontiers in Systems Biology*, 3, June 2023. ISSN 2674-0702. doi: 10.3389/fsysb.2023.1174647.

Núria Folguera-Blasco, Florencia A. T. Boshier, Aydar Uatay, Cesar Pichardo-Almarza, Massimo Lai, Jacopo Biasetti, Richard Dearden, Megan Gibbs, and Holly Kimko. Coupling quantitative systems pharmacology modelling to machine learning and artificial intelligence for drug development: Its pAIns and gAIns. *Frontiers in Systems Biology*, 4, July 2024. ISSN 2674-0702. doi: 10.3389/fsysb.2024.1380685.

Nadia Terranova, Didier Renard, Mohamed H. Shahin, Sujatha Menon, Youfang Cao, Cornelis E.C.A. Hop, Sean Hayes, Kumpal Madrasi, Sven Stodtmann, Thomas Tensfeldt, Pavan Vaddady, Nicholas Ellinwood, and James Lu. Artificial Intelligence for Quantitative Modeling in Drug Discovery and Development: An Innovation and Quality Consortium Perspective on Use Cases and Best Practices. *Clinical Pharmacology & Therapeutics*, 115(4):658–672, 2024. ISSN 1532-6535. doi: 10.1002/cpt.3053.

Sendong Zhao, Chang Su, Zhiyong Lu, and Fei Wang. Recent advances in biomedical literature

mining. *Briefings in Bioinformatics*, 22(3):bbaa057, May 2021. ISSN 1477-4054. doi: 10.1093/bib/bbaa057.

Jiacheng Hu, Runyuan Bao, Yang Lin, Hanchao Zhang, and Yanlin Xiang. Accurate Medical Named Entity Recognition Through Specialized NLP Models, December 2024.

Ting Gao, Xue Zhai, Chuan Yang, Linlin Lv, and Han Wang. Joint extraction of entity and relation based on fine-tuning BERT for long biomedical literatures. *Bioinformatics Advances*, 4(1):vbae194, January 2024. ISSN 2635-0041. doi: 10.1093/bioadv/vbae194.

Jingcheng Du, Dong Wang, Bin Lin, Long He, Liang-Chin Huang, Jingqi Wang, Frank J. Manion, Yeran Li, Nicole Cossrow, and Lixia Yao. Use of deep learning-based NLP models for full-text data elements extraction for systematic literature review tasks. *Scientific Reports*, 15(1):19379, June 2025. ISSN 2045-2322. doi: 10.1038/s41598-025-03979-5.

Ferran Gonzalez Hernandez, Quang Nguyen, Victoria C. Smith, José Antonio Cordero, Maria Rosa Ballester, Màrius Duran, Albert Solé, Palang Chotsiri, Thanaporn Wattanakul, Gill Munding, Watjana Lilaonitkul, Joseph F. Standing, and Frank Klopogge. Named entity recognition of pharmacokinetic parameters in the scientific literature. *Scientific Reports*, 14(1):23485, October 2024. ISSN 2045-2322. doi: 10.1038/s41598-024-73338-3.

Shan He, Yukun Tan, Qing Ye, Matthew M Gubin, Merve Dede, Hind Rafei, Weiyi Peng, Katayoun Rezvani, Vakul Mohanty, and Ken Chen. AI-powered Immune Cell Knowledge Graph (ICKG) with granular immune contexts enables immune program interpretation. *npj Artificial Intelligence*, 2(1):13, January 2026. ISSN 3005-1460. doi: 10.1038/s44387-025-00060-4.

Muhammad Ali Khan, Umair Ayub, Syed Arsalan Ahmed Naqvi, Kaneez Zahra Rubab Khakwani, Zaryab Bin Riaz Sipra, Ammad Raina, Sihan Zou, Huan He, Seyyed Amir Hossein, Bashir Hasan, R. Bryan Rumble, Danielle S. Bitterman, Jeremy L. Warner, Jia Zou, Amye J. Tevaarwerk, Konstantinos Leventakos, Kenneth L. Kehl, Jeanne M. Palmer, M. Hassan Murad, Chitta Baral, and Irbaz Bin Riaz. Collaborative Large Language Models for Automated Data Extraction in Living Systematic Reviews. *medRxiv: The Preprint Server for Health Sciences*, page 2024.09.20.24314108, September 2024. doi: 10.1101/2024.09.20.24314108.

- Aline Gendrin-Brokmann, Eden Harrison, Julianne Noveras, Leonidas Souliotis, Harris Vince, Ines Smit, Francisco Costa, David Milward, Sashka Dimitrievska, Paul Metcalfe, and Emilie Louvet. Investigating deep-learning NLP for automating the extraction of oncology efficacy endpoints from scientific literature. *Intelligence-Based Medicine*, 10:100152, January 2024. ISSN 2666-5212. doi: 10.1016/j.ibmed.2024.100152.
- Anuraag Saini and Ali Farnoud. QSP-Copilot: An AI-Augmented Platform for Accelerating Quantitative Systems Pharmacology Model Development. *CPT: Pharmacometrics & Systems Pharmacology*, 14(11):1775–1786, 2025. ISSN 2163-8306. doi: 10.1002/psp4.70127.
- Enshuo Hsu and Kirk Roberts. LLM-IE: A python package for biomedical generative information extraction with large language models. *JAMIA Open*, 8(2):ooaf012, April 2025. ISSN 2574-2531. doi: 10.1093/jamiaopen/ooaf012.
- Xingbo Wang, Samantha L. Huey, Rui Sheng, Saurabh Mehta, and Fei Wang. SciDaSynth: Interactive Structured Data Extraction From Scientific Literature With Large Language Model. *Campbell Systematic Reviews*, 21(4):e70073, 2025. ISSN 1891-1803. doi: 10.1002/cl2.70073.
- Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017):625–630, June 2024. ISSN 1476-4687. doi: 10.1038/s41586-024-07421-0.
- Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Transactions on Information Systems*, 43(2):1–55, March 2025. ISSN 1046-8188, 1558-2868. doi: 10.1145/3703155.
- Shrey Pandit, Jiawei Xu, Junyuan Hong, Zhangyang Wang, Tianlong Chen, Kaidi Xu, and Ying Ding. MedHallu: A Comprehensive Benchmark for Detecting Medical Hallucinations in Large Language Models, February 2025.
- Yihan Li, Xiyuan Fu, Ghanshyam Verma, Paul Buitelaar, and Mingming Liu. Mitigating Halluci-

nation in Large Language Models (LLMs): An Application-Oriented Survey on RAG, Reasoning, and Agentic Systems, October 2025.

Samuel Colvin, Eric Jolibois, Hasan Ramezani, Adrian Garcia Badaracco, Terrence Dorsey, David Montague, Serge Matveenko, Marcelo Trylesinski, Sydney Runkle, David Hewitt, Alex Hall, and Victorien Plot. Pydantic Validation, October 2025. URL <https://github.com/pydantic/pydantic>.

Hernán E. Grecco. Pint: Operate and manipulate physical quantities in Python, 2025. URL <https://github.com/hgrecco/pint>.

Pydantic. Logfire: Observability platform for Python applications, 2025. URL <https://pydantic.dev/logfire>.

Hong Ge, Kai Xu, and Zoubin Ghahramani. Turing: A Language for Flexible Probabilistic Inference. In *Proceedings of the Twenty-First International Conference on Artificial Intelligence and Statistics*, pages 1682–1690. PMLR, March 2018.

Christopher Rackauckas and Qing Nie. Differentialequations.jl – a performant and feature-rich ecosystem for solving differential equations in julia. *The Journal of Open Research Software*, 5(1), 2017. doi: 10.5334/jors.151. URL <https://app.dimensions.ai/details/publication/pub.1085583166> and <http://openresearchsoftware.metajnl.com/articles/10.5334/jors.151/galley/245/download/>. Exported from <https://app.dimensions.ai> on 2019/05/05.

Mathieu Besançon, Theodore Papamarkou, David Anthoff, Alex Arslan, Simon Byrne, Dahua Lin, and John Pearson. Distributions.jl: Definition and modeling of probability distributions in the juliastats ecosystem. *Journal of Statistical Software*, 98(16):1–30, 2021. ISSN 1548-7660. doi: 10.18637/jss.v098.i16. URL <https://www.jstatsoft.org/v098/i16>.

Saibo Geng, Hudson Cooper, Michał Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. JSONSchemaBench: A Rigorous Benchmark of Structured Outputs for Language Models, February 2025.

Tomasz Pawłowski, Grzegorz Bokota, Georgia Lazarou, Andrzej M. Kierzek, and Jacek Sroka. Emulation of Quantitative Systems Pharmacology models to accelerate virtual population inference

762 in immuno-oncology. *Methods (San Diego, Calif.)*, 223:118–126, March 2024. ISSN 1095-9130.
763 doi: 10.1016/j.ymeth.2023.12.006.